

Técnicas de diseño de casos de prueba

¿Por qué no se puede probar «todo»?

Probar todas las combinaciones posibles de entrada es imposible. Cuanto más crece el software, más astronómico se vuelve el número de casos.

2 números
0 a 99
10.000 casos



Formulario
10 campos de texto
millones de casos



Aplicación real
miles de variables
imposible cubrir todo

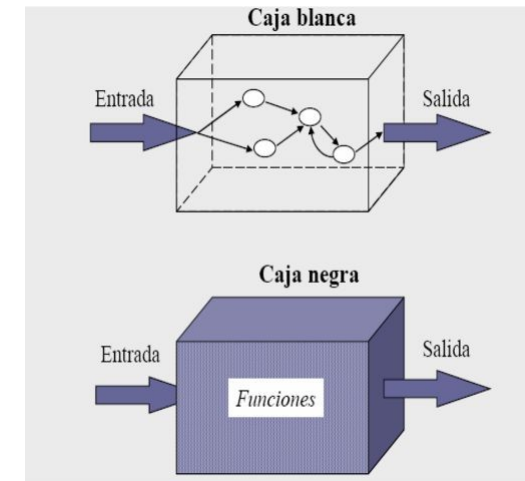
La solución es elegir un conjunto representativo: casos que maximizan la detección de errores con el menor número de pruebas posible.

Caja blanca

Examina el código por dentro.
Prueba rutas lógicas, condiciones y bucles.

Caja negra

Solo mira entradas y salidas.
Prueba según la especificación.



Pruebas de caja blanca

Las pruebas de caja blanca se centran en la estructura interna del programa: sus rutas lógicas, condiciones y bucles. El probador tiene acceso al código fuente y lo examina directamente.

¿Qué se analiza?

- **Rutas de ejecución**
 - Qué caminos puede tomar el programa según las condiciones del código.
- **Condiciones y decisiones**
 - Que todas las ramas de un if o un switch se evalúan correctamente.
- **Bucles**
 - Si se ejecutan el número de veces esperado, y qué pasa si no se ejecutan ninguna.

Pruebas de caja blanca — cobertura y valoración

Cobertura del código

Mide qué porcentaje del programa se ha ejercitado realmente:

Métrica	Qué mide
Cobertura de sentencias	¿Se ejecutó cada línea al menos una vez?
Cobertura de condiciones	¿Se evaluaron todos los valores posibles de cada condición?
Cobertura de rutas	¿Se probaron todas las combinaciones de caminos posibles?

Ventajas e inconvenientes

A favor

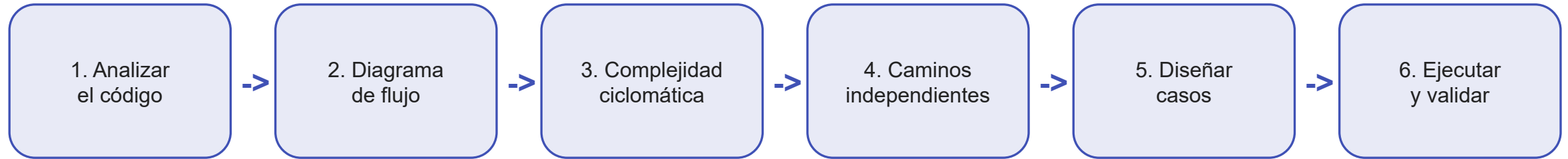
- Detecta errores de lógica interna que la caja negra pasaría por alto
- Identifica código innecesario o redundante

En contra

- Exige conocer el código a fondo
- Consume más tiempo que la caja negra
- En sistemas grandes, cubrir todas las rutas puede ser impracticable

La técnica del camino básico

Forma sistemática de aplicar la caja blanca: identifica todas las rutas independientes de un programa para garantizar que se cubre toda su lógica interna.



Complejidad ciclomática — $M = E - N + 2$

E = número de aristas N = número de nodos M = número de caminos independientes

M es el número mínimo de casos de prueba necesarios para cubrir toda la lógica del método.

Ejemplo — calcularTotalConDescuento: 7 nodos, 8 aristas → $M = 8 - 7 + 2 = 3$ → 3 casos de prueba mínimos.

La técnica del camino básico — los 6 pasos

Paso	Qué se hace	Qué se produce
1. Analizar el código	Leer el código e identificar todas las estructuras de control: if, for, while, switch...	Lista de puntos de decisión y bifurcaciones del método
2. Diagrama de flujo	Asignar un nodo a cada punto relevante y dibujar las aristas que los conectan	Grafo de flujo de control con nodos etiquetados y aristas dirigidas
3. Complejidad ciclomática	Contar nodos y aristas del grafo y aplicar $M = E - N + 2$	Número M: indica cuántos caminos independientes existen
4. Caminos independientes	Listar los M caminos que recorren el grafo sin repetir rutas ya cubiertas	Lista C1, C2... Cm con el recorrido de nodos de cada camino
5. Diseñar casos de prueba	Para cada camino, elegir una entrada concreta que lo fuerce y anotar la salida esperada	Tabla de casos con entrada, camino cubierto y salida esperada
6. Ejecutar y validar	Ejecutar el código con cada entrada y comparar la salida real con la esperada	Confirmación de que todos los caminos funcionan correctamente

Ejemplo — calcularTotalConDescuento

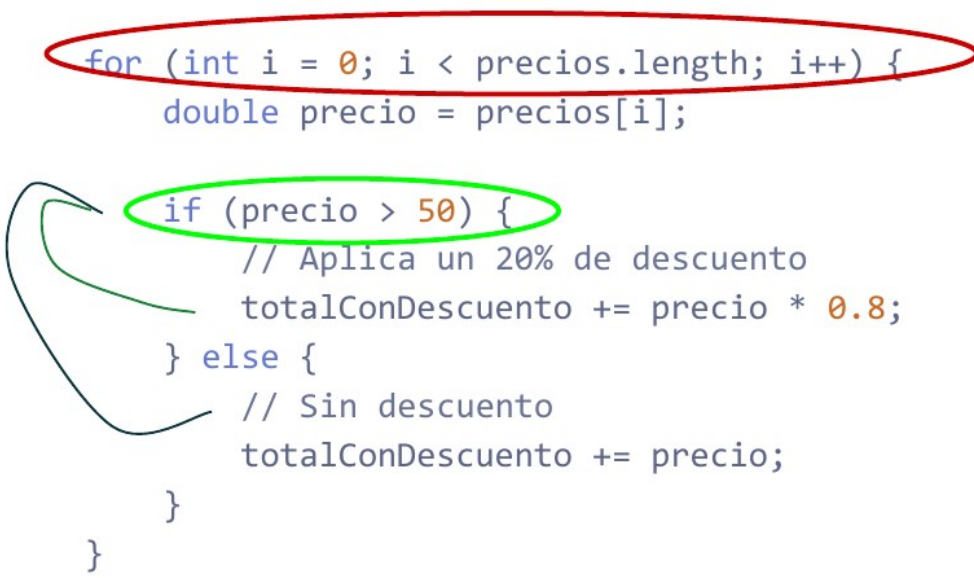
El método recibe un array de precios y aplica un 20 % de descuento a los que superen 50 €.

```
public static double calcularTotalConDescuento(double[] precios) {  
    double totalConDescuento = 0;  
  
    for (int i = 0; i < precios.length; i++) {  
        double precio = precios[i];  
  
        if (precio > 50) {  
            // Aplica un 20% de descuento  
            totalConDescuento += precio * 0.8;  
        } else {  
            // Sin descuento  
            totalConDescuento += precio;  
        }  
    }  
  
    return totalConDescuento;  
}
```

Paso 1 — Identificar estructuras de control

Se marcan el bucle for (recorre el array) y el if/else (bifurca según el precio de cada elemento).

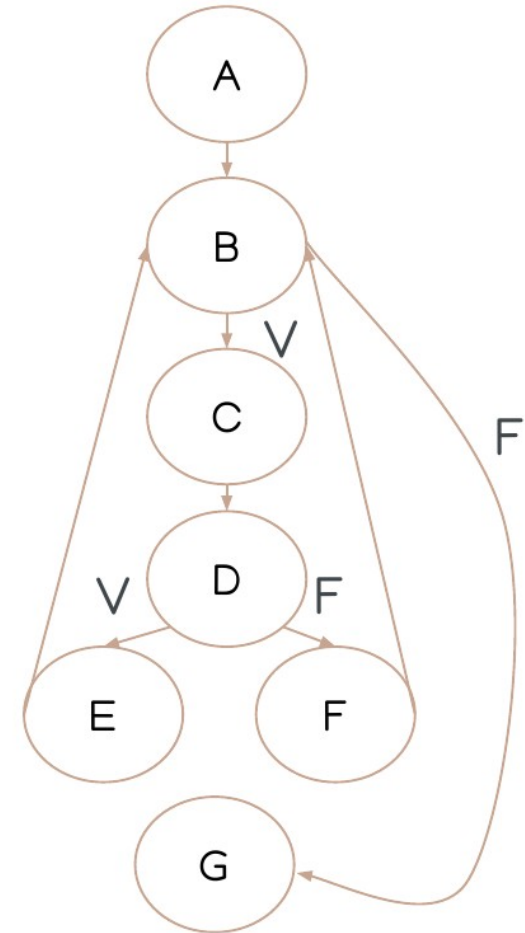
```
public static double calcularTotalConDescuento(double[] precios) {  
    double totalConDescuento = 0;  
  
    for (int i = 0; i < precios.length; i++) {  
        double precio = precios[i];  
  
        if (precio > 50) {  
            // Aplica un 20% de descuento  
            totalConDescuento += precio * 0.8;  
        } else {  
            // Sin descuento  
            totalConDescuento += precio;  
        }  
    }  
  
    return totalConDescuento;  
}
```



Paso 2 — Diagrama de flujo de control

A cada punto relevante se le asigna un nodo (A–G) y se dibujan las aristas. La arista G→B representa la vuelta al bucle.

```
public static double calcularTotalConDescuento(double[] precios) {  
    A double totalConDescuento = 0;  
    B for (int i = 0; i < precios.length; i++) {  
        C double precio = precios[i];  
        D if (precio > 50) {  
            // Aplica un 20% de descuento  
            E totalConDescuento += precio * 0.8;  
        } else {  
            F // Sin descuento  
            totalConDescuento += precio;  
        }  
    }  
    G return totalConDescuento;  
}
```



Paso 3 — Complejidad ciclomática

Con el grafo listo se aplica $M = E - N + 2 = 8 - 7 + 2 = 3$.

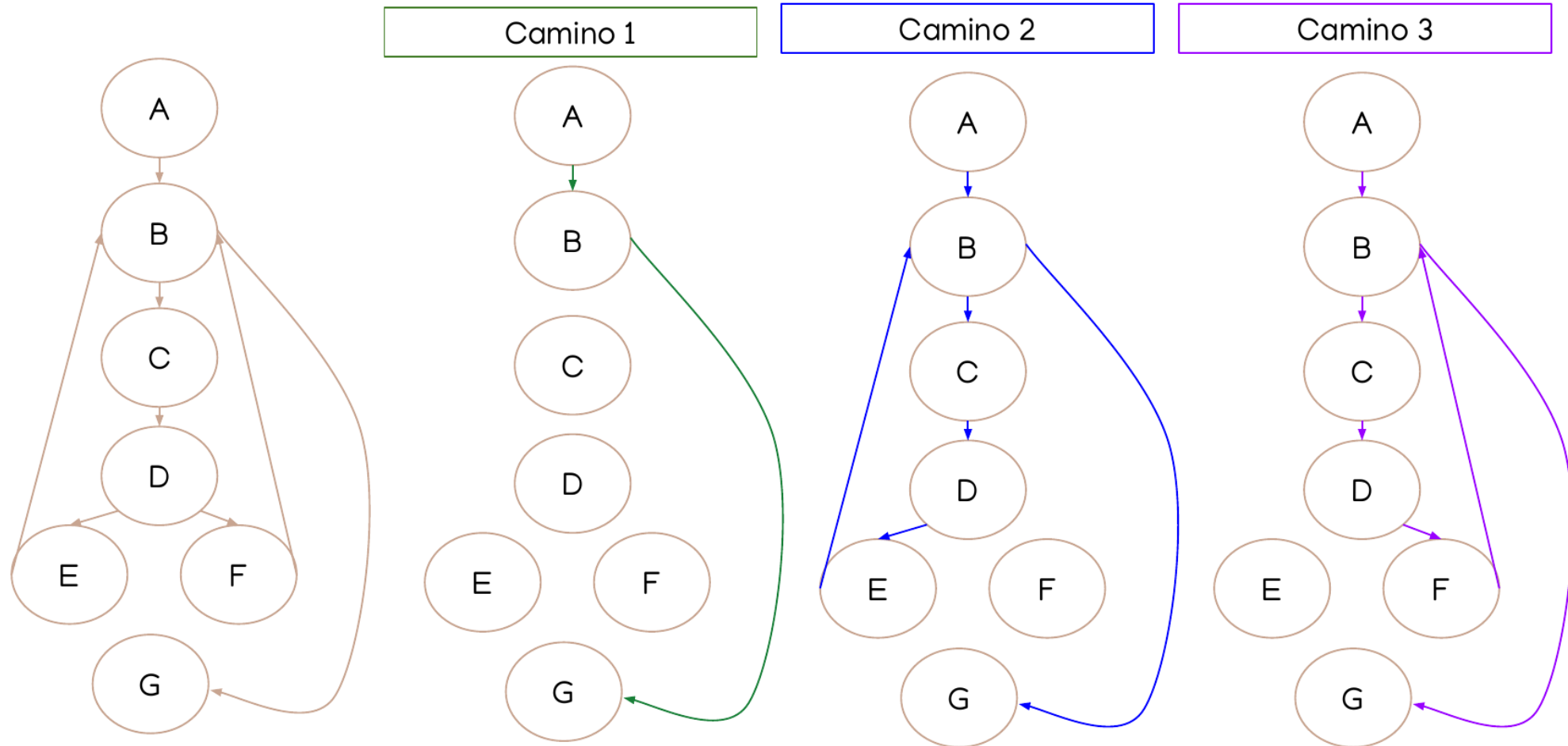
Hay 3 caminos independientes, así que hacen falta al menos 3 casos de prueba.

Los tres caminos son: C1 (A→B→G, array vacío), C2 (A→B→C→D→E→G, precio > 50) y C3 (A→B→C→D→F→G, precio ≤ 50).

En la siguiente diapositiva se muestran los grafos coloreados.

Paso 4 — Caminos independientes

El grafo completo a la izquierda; a la derecha, cada uno de los 3 caminos resaltado con su color.



Paso 5 — Diseñar casos de prueba

Un caso por camino: se elige una entrada concreta y se anota la salida esperada.

Camino 1

- Descripción: Array vacío
- Entrada: {}
- Salida esperada: 0.0

Camino 2

- Descripción: Precios mayores a 50 (descuento)
- Entrada: {60.0, 100.0}
- Salida esperada: $60 \times 0.8 + 100 \times 0.8 = 128.0$

Camino 3

- Descripción: Precios menores o iguales a 50
- Entrada: {20.0, 50.0}
- Salida esperada: $20 + 50 = 70.0$

Paso 6 — Ejecutar y validar

Se ejecutan los 3 casos y se comprueba que la salida real coincide con la esperada.

Camino 1

- **Descripción:** Array vacío
- **Entrada:** {}
- **Salida esperada:** 0.0

Camino 2

- **Descripción:** Precios mayores a 50
- **Entrada:** {60.0, 100.0}
- **Salida esperada:** $60 \times 0.8 + 100 \times 0.8 = 128.0$

Camino 3

- **Descripción:** Precios menores o iguales a 50
- **Entrada:** {20.0, 50.0}
- **Salida esperada:** $20 + 50 = 70.0$

```
La entrada es []  
El precio final para el camino es 0.0  
La entrada es [60.0, 100.0]  
El precio final para el camino es 128.0  
La entrada es [20.0, 50.0]  
El precio final para el camino es 70.0
```

Pruebas de caja negra

Evalúan el comportamiento del software a partir de sus especificaciones, entradas y salidas, sin mirar cómo está implementado por dentro. Son complementarias a la caja blanca, no una alternativa.

¿Qué tipos de errores detecta?

- Funciones incorrectas o ausentes
- Errores de interfaz
- Errores en estructuras de datos o en accesos a bases de datos externas
- Errores de rendimiento
- Errores de inicialización y terminación

Los casos de prueba se diseñan atendiendo a las especificaciones del problema, sin importar los detalles internos del programa.

Técnicas de caja negra

Hay tres técnicas principales para diseñar casos de prueba de caja negra:

Partición Equivalente (PE)

Agrupar las entradas en clases de equivalencia.

Todos los valores de una misma clase deberían comportarse igual.

Basta probar un representante por clase.

Análisis de Valores Límite (AVL)

Complementa la PE probando los bordes de cada clase.

Los errores se concentran ahí: un $<$ que debería ser $\leq$, un índice fuera de rango...

Solo tiene sentido con rangos numéricos o conjuntos finitos.

Pruebas Aleatorias

Genera entradas al azar para descubrir errores en combinaciones no previstas.

Útil cuando no hay un patrón concreto que seguir o el espacio de entradas es muy grande.

Partición equivalente — directrices

Para cada condición de entrada, las clases de equivalencia se determinan según el tipo de entrada:

Tipo de entrada	Clases válidas	Clases inválidas
Rango de valores ej. [20..30]	1 (un valor dentro, p. ej. 25)	2 (uno por debajo y otro por encima: 15 y 40)
Conjunto finito de valores ej. {2, 4, 6, 8}	1 (un valor del conjunto, p. ej. 4)	2 (uno fuera por debajo y otro por encima: 1 y 10)
Condición booleana (T/F) ej. «debe ser una letra»	1 (un valor que la cumple, p. ej. «j»)	1 (un valor que no la cumple, p. ej. «?»)
Conjunto de valores admitidos ej. {opción1, opción2, opción3}	tantas como valores admitidos (3 en el ejemplo)	1 (un valor no admitido)

Las clases inválidas se cubren de una en una para poder aislar el error. Las válidas pueden cubrirse varias en el mismo caso de prueba.

Partición equivalente — ejemplo (pasos 1 y 2)

Validar la edad de un usuario: debe estar entre 18 y 65 años (ambos inclusive).

Paso 1 — Identificar las clases

Entrada	Tipo de datos	Clases válidas	Clases inválidas
edad	rango de valores [18..65]	1	2

Paso 2 — Tabla de clases de equivalencia

Condición	Clases válidas	Clases no válidas
edad	(1) valor dentro de [18..65]	(2) valor por debajo de 18 (3) valor por encima de 65

Partición equivalente — ejemplo (paso 3)

Paso 3 — Diseñar los casos de prueba

Un caso por cada clase de equivalencia identificada (válidas e inválidas):

Caso de prueba	Clases cubiertas	Salida esperada
edad = 33	(1) válida	Aceptado
edad = 10	(2) inválida — por debajo de 18	Rechazado
edad = 80	(3) inválida — por encima de 65	Rechazado

Análisis de valores límite (AVL)

Los errores se concentran en los límites: un $<$ que debería ser $\leq$, un índice fuera de rango... El AVL complementa la PE probando específicamente esos bordes.

Para la entrada edad con rango [18..65]:

Condición	Clases válidas en el límite	Clases inválidas junto al límite
edad	(1) 18 (2) 19 (3) 64 (4) 65	(5) 17 (6) 66

Casos de prueba AVL — los 6 valores críticos

Caso	Valor	Clases cubiertas	Salida
Límite inferior	18	(1)	Aceptado
Adyacente interior inf	19	(2)	Aceptado
Adyacente interior sup	64	(3)	Aceptado
Límite superior	65	(4)	Aceptado
Justo fuera (inf)	17	(5)	Rechazado
Justo fuera (sup)	66	(6)	Rechazado

Pruebas aleatorias

Cuando no se sigue ningún patrón concreto, se pueden generar entradas al azar para descubrir errores en combinaciones que no se habrían ocurrido diseñando casos a mano.

Ejemplo — calculadora con cuatro operaciones

Para una calculadora que acepta dos números y una operación (+, -, x, /):

Entrada 1	Entrada 2	Op.	Por qué es interesante
25	5	+	Caso genérico, sin valores límite
-12	0	/	Detecta si el programa controla la división por cero
9	-3	x	Multiplicación con un número negativo
100	50	-	Resta con resultado positivo grande